

Cloud Final Report Grp 10

Online Learning Platform for MOE

Muhammad Adli Bin Jumahat (2401516)

Subhashini Rajkumar(2402769)

Triona Lee Puay Ki (2403478)

Swathi Ravichandran (2402695)

Borse Srushti Ganesh (2401618)

Lai Yueyin Shyann (2402456)

1. Abstract

This paper details the architecture and function of a proposed digital learning platform designed for secondary education students in Singapore, with the goal of resolving critical issues related to resource distribution, data vulnerability, and passive student engagement. The system establishes a private, collaborative, and paperless educational ecosystem, structured around a decentralized storage model. Utilizing cloud-native principles, the platform ensures elastic scalability and provides robust data isolation. Thus, the system is designed to be technically resilient, interactive and functionally advanced, effectively aligning with the Ministry of Education's vision for digital learning.

2. Introduction

Singapore's Ministry of Education (MOE) has made digital learning a priority, but its implementation in secondary schools is not always consistent. Many schools depend on a mix of third-party cloud services like Google Classroom and Microsoft OneNote, which can create problems. These issues include scattered resources and increased risks to data security, as academic information is stored on external systems. Past attempts to use centralized platforms, such as MC Online, often had poor user experience and failed to engage students effectively. Simply providing static documents or PDFs does not take full advantage of what digital environments can offer. Without interactive tools, online learning can become just a place to store content, instead of a place that promotes thinking and engagement. This problem is what the proposed solution aims to fix.

3. Proposed System Contribution and Significance

The proposed system is a strong, cloud-based solution designed to solve these problems. A key feature is its decentralized storage model, which keeps data separate and secure. By dividing storage into School Storage and individual Student Storage, the platform provides a paper-free learning environment and ensures privacy. The platform also supports collaborative tools like the live Whiteboard and the Flashcard Creator. These tools turn static learning materials into shared, dynamic workspaces, encouraging group

problem-solving and peer assessment. In addition, by using containerization and Kubernetes for its infrastructure, the platform can quickly adjust to handle high demand and remain reliable through replication and failover. It results in a technologically strong and secure platform that helps achieve the MOE's goals for advanced digital learning.

3.1 System Design:

Describe your design choices and give a high-level overview of your system and its functionality; include figures and/or diagrams to explain concepts, interactions between components etc.; justify your design choices and discuss any resulting implications

3.1.1 Frontend

The project is a multi-page website built with plain HTML, CSS, and JavaScript, deployed as static files via Dockerized Nginx and intended for easy use with Firebase Hosting. The simple structure features separate pages for Authentication and Dashboards (Student/Teacher) and includes tools like a Document Editor, Flashcards, and Whiteboard. State is managed on the client using LocalStorage for basic session info and in-memory JS state for dynamic data. The frontend communicates with a specialized backend architecture: a REST API for common actions (login, data loading), a Flashcard Game Server for real-time competitions, and a Whiteboard Server for live drawing, ensuring clear separation of services, easy scaling, and fast loading times due to the static file approach.

3.1.2 Backend Cloud Firebase and Cloud Storage

Firestore was chosen as it's a fully-managed, serverless database that scales automatically to handle increasing load. Its built-in real-time synchronization is essential for supporting the collaborative features ensuring all students are able to see updates instantly. The platform supports strict data separation via data partitioning. This design choice allows the platform to run from a single central database while enforcing data privacy.

Cloud Storage is also being used to store all uploaded lesson materials, exported files, and student copies. It works alongside the database, which keeps track of the file information such as who owns it and where it belongs in the course. The backend API (running on Cloud Run) sits

between the frontend and these services. It decides where files should go, generates temporary upload/download links, and updates the database. Browser clients never access storage directly; instead, they upload, download and view files using the short-lived links provided by the backend. Security is maintained by using temporary links and backend-controlled access rules.

3.1.3 Whiteboard

The whiteboard system provides a real-time collaborative whiteboard that is integrated to the document platform. Each document has its own whiteboard room where multiple users can draw simultaneously on the same whiteboard room (refer to Fig. 3.1.3).

Fabric.js was chosen to implement freehand drawing. The library also has the ability to serialize and deserialize the strokes or paths as JSON, making it easy for us to store the white-board strokes into the firestore database. Providing dedicated whiteboard rooms helps with isolation between collaboration groups and prevents cross-document interference. Firestore database helps with the whiteboard load and save strokes which helps in data consistency. The Firestore database also helps by ensuring all whiteboard rooms are valid by cross referencing with the database for the same document id.

3.1.4 Flashcards

The Flashcard application is built on a Hybrid Three-Tier Architecture that combines standard data management with real-time interaction. The system is composed of the Client (JavaScript pages), the Application Server (Node.js/Express.js), and the Data Access Layer (Firestore). This hybrid design employs two distinct communication protocols: RESTful APIs are used for asynchronous, reliable deck management (CRUD), while WebSockets (socket.io) handle the low-latency, bi-directional communication necessary for the real-time competition feature.

This architecture prioritizes both security and performance. Data Isolation is achieved through Server-Side Authentication, where the client's x-user-id header forces the server to construct a user-specific path in Firestore (e.g., /users/{userId}/decks), ensuring private and categorized data access. For competitions, the use of socket.io allows the server to maintain a Server-Authoritative State in memory, which resolves concurrency conflicts and instantly synchronizes scores between players. On the

front-end, the server's View Routes manage the loading of separate HTML pages, simulating an SPA-like experience that simplifies client-side development.

3.1.5 Cloud Deployment: Docker

The platform uses a container-based architecture to ensure scalability, reproducibility, and controlled deployment across schools. Every subsystem is packaged as an isolated Docker image and stored in a centralized Artifact Registry.

Its architecture consists of three services. The Frontend Client provides the user interface, integrating document viewing, flashcards, and whiteboard sessions. The Backend/API server manages authentication, storage operations, metadata, and inter-service coordination. The Realtime collaboration service supports low-latency realtime sessions through Websockets.

This design permits independent scaling, secures isolated user data, and ensures that service failures are contained.

3.1.6 Cloud Deployment: Cloud Run

We have decided to orchestrate Whiteboard, Flashcard and Backend containers on Cloud Run. This is beneficial for our application as Cloud Run allows for fast autoscaling, is pay-per-use, and has built in security. These factors make our application fault tolerant and highly scalable in a secure manner.

The frontend is hosted on Firebase, which uses Content Delivery Network, allowing for faster load times. The frontend client is the only user-facing component and it interacts with the other 3 backend servers - backend, whiteboard and flashcard, which require authentication.

3.2 Implementation:

Describe the details of your implementation; how you have realised your design; what aspects of it worked as expected and whether there has been any missing functionality; how you have dealt with important issues such as concurrency, server-to-client communication, fault and failure handling, etc.

3.2.1 Frontend

This is a lightweight, framework-free multi-page website hosted on Firebase. It uses plain HTML/CSS/JS and manages API URLs via a configuration file. User sessions are stored locally and verified upon loading. The site features a document editor (Quill/Fabric.js) which uses a

REST API and Firestore for syncing. It also includes real-time flashcard competitions (Socket.IO) and a collaborative whiteboard (WebSockets).

3.2.2 Backend Cloud Firebase and Cloud Storage

The database implementation supports multi-tenancy by structuring data hierarchically into Collections and Documents. The Firebase Admin SDK was used to implement the server-side logic. Application Default Credentials, which are automatically provided by the Google Cloud Shell environment, were used by the code to initialize the SDK. This allows the Node.js script to correctly fetch data from the Firestore collection and authenticate. The Firestore platform manages shared data states and transactions ensure that all changes occur together or not at all, preventing inconsistencies. The implementation includes explicit try-catch blocks around database operations which handle runtime faults like network timeouts or permission errors.

3. Cloud Storage Implementation

Google Cloud Storage (GCS) is wired through the backend, which organizes files by user and context. To manage file access, the backend generates short-lived Signed URLs: PUT links for direct uploads from the frontend and read links for viewing. The standard upload flow involves the frontend requesting an upload URL, uploading the file directly to GCS, and the backend saving the storage path in Firestore. Sharing documents uses an atomic server-side object copy operation (copy-to-student) to ensure isolated copies for each user. Concurrency is managed by the unique, time-limited signed URLs.

3.2.3 Whiteboard

The Whiteboard system can be separated into client implementation, server implementation and firestore database integration.

The client implementation starts with the html initialising the Fabric canvas and loading the document information. The drawing component is disabled until the client is able to communicate a response from the server that the room ID is valid and that the user is allowed to be in the whiteboard room.

For the server, when a client connects, the server reads the roomId and checks Firestore for its validity. If the room exists, the server retrieves any saved strokes and sends them to the whiteboard clients. After receiving either the existing strokes

or the initialization response, the client enables the drawing component, allowing the user to start drawing.

A client stroke triggers an event from the server which causes the server to save the latest stroke onto the whiteboard-strokes collection in the firebase db under that document id. At the same time, it will also broadcast to all other clients in the same room to remain in sync.

When the server receives clear data, the server will delete all the stroke documents inside that document id's whiteboard-strokes collection.

For DB integration, Under each document id, when the user enters a whiteboard and draws a stroke, the server will create a stroke document, containing the path json data, under the the whiteboard-strokes collections. If the collection does not exist, the collection will be created under the document id, making the board persistent.

For Future Enhancement, We can enhance the Undo or Redo strokes functionality to have better control if the user wants to remove the newly added strokes or re-add the newly removed strokes. We can implement stroke editing functionality. Being able to select, move or delete on existing strokes makes the user have better control on the whiteboard canvas. Adding different permissions such as view only permissions will allow for better control.

For handling major issues, for currency, the whiteboard server had to manage incoming data per room and broadcast to all clients only in the same room. Each user stroke had to be saved to the firestore database so for the server to be able to load the whiteboard consistently across all clients. The clear operation needs to happen in a batch commit to avoid partial deletion between clients. The server needs to check if the connecting client is authorized or not. If not, redirect with a generic alert message.

For Fault Tolerance, on the client side, the whiteboard functionality remains disabled until the client receives data from the server to acknowledge that the whiteboard is fully loaded. This prevents the user to start drawing before the strokes is loaded in or prevents an unauthorised user to draw during the moments before he gets redirected

On the server side, if the Firestore validation fails, close the connection to prevent unauthorised access and redirect back to login.

On the database side, the firestore database holds the stroke history for each whiteboard room to

load from which enables the whiteboard to be persistent.

3.2.4 Flashcards

1. Implementation Details and Design

The flashcards section is built on a 3 tier architecture: Presentation (Client), Application (Express/Socket.io) and Data (Firestore). It is split into 3 separate directories:

frontend\public\flashcards for the main client files, frontend\src\routes for the server-side API/views, and realtime\flashcards for the server entry point.

Client-Side Design (frontend): a multi-page web application managed by the server's view routes

- The Student Dashboard serves as the initial hub, where the client first validates the user session using the sessionUser object stored in localStorage. Once authenticated, the dashboard loads the user's navigational structure and delegates subject data management to the client-side (student.js). This script uses the openSubject(subjectId) function to establish the Subject-Based Context, which is required to organize the decks before the user navigates to the flashcard-specific views (CRUD, Study, Competition). Crucially, to enforce data isolation on the backend, a custom x-user-id HTTP header containing the user's identity (from localStorage) is attached to every API request.

Server-Side Design (realtime\flashcards, frontend\src\routes): The server is built on Express.js and Socket.IO

- The RESTful API handles all deck management (CRUD) under the /api/decks endpoint. This API is designed for data security and categorization by strictly including the User ID and Subject ID in the Firestore collection path: flashcardSets/{subjectId}/users/{userId}/decks/{deckId}. For the client interface, View Routing relies on server-side routes (views.js) to load separate HTML pages for each function (list, editor, study, competition). This gives the user the experience of a single, continuous application.

2. Functionality Analysis

Aspects That Worked as Expected (Design Goals Met): The application successfully implements the core features, achieving full CRUD (Create, Read, Update, Delete) management for both decks and

individual flashcards through client-side scripts like new_flashcard.js and all_decks.js. For advanced features, the use of socket.io establishes a functional, low-latency, two-way communication channel, which serves as the foundation for the Real-Time Competition, enabling synchronized deck sharing and state updates (refer to Fig. 3.2.4). Throughout all operations, API calls on the client side are protected by try catch blocks, utilizing custom modal windows to provide users with clear, non-disruptive feedback on success or failure, enhancing the overall user experience and making the system more robust.

The project overcame significant technical hurdles, starting with the Data Transition from local storage to a persistent backend using a dedicated RESTful API. This transition established a critical client-server contract where the client's x-user-id HTTP header is used to construct the correct, isolated Firestore path for every CRUD operation, ensuring data privacy and correct categorization. A major design issue was implementing two distinct Dual Study Logics for card looping: the Study Mode repeats cards marked as Wrong or Skip (moving them to the array end), whereas the Competition Mode only returns cards marked Wrong, requiring full deck mastery to finish. Concurrency in the real-time competition is managed by enforcing a Server-Authoritative State via Socket.IO, which centrally processes actions and resolves conflicts. This Hybrid Communication Strategy (REST for management, WebSockets for real-time state) is fortified by robust Fault Handling on both the client (custom modals) and server (try catch blocks) to maintain application stability.

3.2.5 Docker

The system deploys four containerized components, built with multi-stage Dockerfiles and stored in a secure Artifact Registry. This structure enables clean separation, controlled deployment, and independent scaling.

The Frontend Client Container serves the compiled web application through a lightweight static server (nginx). Meanwhile, Backend/API Server Container implements authentication, storage APIs, CRUD operation, and business logic for efficient load balancing. The Realtime Server Containers provide synchronous collaboration using WebSockets and Redis pub/sub for state consistency across replicas.

For the CI/CD and Deployment Pipeline, CI builds and tests all container images, pushing them with SHA-based tags its rolling updates ensure zero-downtime deployment liveness and readiness probes prevent unhealthy instances from serving traffic

Realtime operations are handled via WebSockets with Redis backing and the API uses non-blocking request handling and database connection pooling.

3.2.6 Cloud Run

All services are configured with 0 minimum instances to ensure cost control and the Whiteboard Server uses WebSockets to maintain persistent connections. The frontend is connected to the servers on Cloud Run via their public HTTPS service URLs. When the user performs an action (e.g. saving flashcards), the frontend sends API requests or WebSocket connections directly to the Cloud Run endpoints. Firebase Hosting serves the static site globally, and all dynamic operations are handled by the Cloud Run services. The backend service is deployed with a Service Account, which is granted a role required for database operations on Cloud Datastore Use. This ensures the application code can securely perform database reads and writes via Application Default Credentials (ADC) without needing to embed sensitive passwords or key files in the container.

3.3 Evaluation:

The table (Fig. 3.3.1) confirms that when the service is not in use, it correctly scales down, resulting in minimal to zero cost during idle times. Services correctly auto-scale to handle the high-throughput. The instances only run when needed. All three services maintain a P95 latency (Fig. 3.3.2) consistently under 200ms.. This proves that even during traffic spikes, we are experiencing responsive interactions. Brief spikes indicate a "cold start" event where an instance had to spin up from zero. The CPU usage (Fig. 3.3.3) for all services is low, generally hovering below 10%. The low traffic on (Fig. 3.3.4) the Flashcard service proves it is better to have it as a separate service, saving substantial cost compared to being bundled with the high-traffic Whiteboard service.

3.4 Related Work:

Blackboard provides a single environment for administration, content delivery, and group interaction. Blackboard (Wu, 2021) excels in facilitating group communication through integrated tools like video chat, forums, calendars,

and discussion boards. While Blackboard supports forums and video chat, it lacks a synchronous collaborative whiteboard.. Specialized tools, like Gizmo (Gizmo, 2025) , focus heavily on individual self-study. Our system introduces a Real-Time Competition Mode for flashcards, turning a solitary learning activity into a competitive group engagement. LMS, like Moodle (Dagger, 2007), rely on monolithic architectures that are expensive to run. Our architecture ensures that idle services consume zero resources.

4. Conclusions:

4.1 Frontend - Subhashini

I designed the page flows and UI behavior, implemented secure login/sign-up, built the signed URL-based file upload pipeline, and created the dashboards and editors for document management and export. My focus was on maintaining a secure, modular, and extensible system.

4.2 Backend Cloud Firebase - Swathi

My main contribution was to implement the Firestore Database and connect it to the backend to ensure data can be saved, retrieved and manipulated with no security breaches. My main take-away is that implementing a secure, scalable application on Firebase requires a clear separation between client-side limitations and server-side privilege (managed by the Admin SDK). Future work should focus on cost optimization by migrating Admin SDK logic to serverless Firebase Callable Cloud Functions and enhancing real-time performance for collaboration features.

4.3 Backend Cloud Storage - Subhashini

My primary contributions involved designing the storage structure, building the signed-URL pipeline (uploads/views), and guaranteeing atomic copy operations for student files. The system leverages signed URLs to delegate file transfer to the client, achieving high efficiency and scalability with low server overhead.

4.4 Docker - Shyann

My main contribution was designing a secure and repeatable deployment pipeline using Docker and Artifact Registry, with scalable microservices orchestrated by Cloud Run. My key takeaway is a

deeper understanding of practical cloud-native principles, such as containerisation, service decoupling, and automated deployments that translates into real operational reliability. It strengthened my ability to design systems that are functional and scalable in production environments. Could introduce health-based canary rollouts and automated rollback triggers in the possible future to improve deployment reliability during updates.

4.5 Cloud Run - Srushti

I orchestrated the Dockerized services using Cloud Run, managing secure service-to-service communication. I would improve my deployments by implementing a Virtual Private Cloud (VPC) for the backend, improving privacy. I would implement auth middleware in the server code to explicitly verify and decode the Google ID Token. This can be coupled with IAMs on Cloud Run for enhanced security. I learnt how services like Cloud Run simplify deployment by handling infrastructure and scaling automatically.

4.6 Whiteboard - Adli

The whiteboard system feature provides a functional, real-time collaborative whiteboard that integrates with a document-based database for persistence. The main takeaway is to create features in phases. In phase 1, I created a basic whiteboard that supported real-time concurrent drawing among clients. In the next phase, I added proper collaboration groups, prevented

cross-document interference, integrated Firestore for persistence, and implemented user ID validation so only authorized users could access each whiteboard. By implementing the feature in smaller phases, I was able to progressively enhance the whiteboard to its current state. In future phases, I would continue setting small, focused goals to further improve the user experience.

4.7 Flashcards - Triona

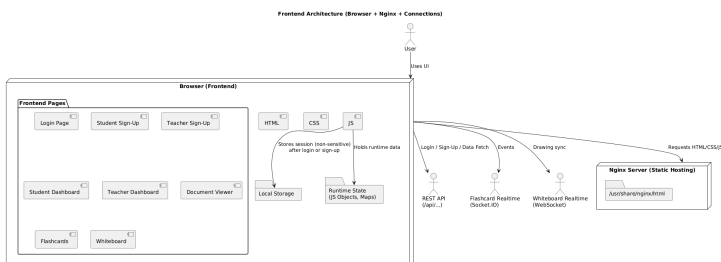
The Flashcard section is implemented as a Hybrid 3-Tier System to manage the full deck lifecycle. Its core design uses a dual communication approach: RESTful APIs handle reliable, standard CRUD operations, while WebSockets enable low-latency, bi-directional communication for the real-time competition feature. Security is guaranteed through Data Isolation; the client passes an x-user-id header which the server uses to scope all Firestore database queries to a user-specific collection path, ensuring data privacy and correct subject categorization. The main takeaway from the flashcard implementation is the completed integration of REST and WebSocket within the entire application architecture to deliver scalable data management and real-time multiplayer collaboration. To boost the app's educational value and collaborative utility, future enhancements should include replacing the flashcards' basic linear review with an enhanced learning algorithm and introducing secure deck sharing to generate cloneable links for content exchange.

5. References:

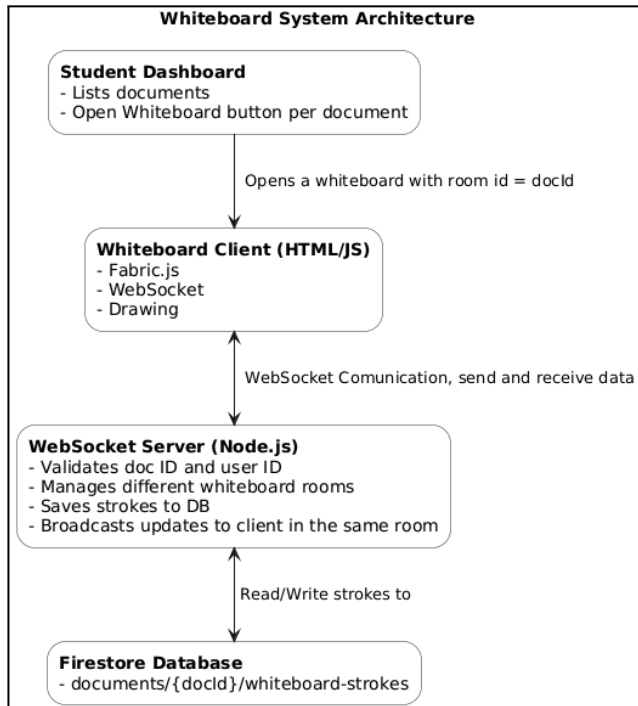
1. E-Learning Based on Cloud Computing. (2021). *LearnTechLib*.
<https://www.learntechlib.org/p/220086/?nl=1>
2. *Gizmo*. (n.d.). GizmoAI. Retrieved November 18, 2025, from <https://gizmo.ai/>
3. Kai, N. W. (2020, December 30). Secondary school, junior college students will spend 2 days a month doing home-based learning from next year. *The Straits Times*.
<https://www.straitstimes.com/singapore/secondary-school-junior-college-students-to-spend-2-days-a-month-doing-home-based-learning>
4. Service-Oriented E-Learning Platforms: From Monolithic Systems to Flexible Services. (2007). *ResearchGate*.
https://www.researchgate.net/publication/3419928_Service-Oriented_E-Learning_Platforms_From_Monolithic_Systems_to_Flexible_Services#:~:text=ELEARNING%20PLATFORMS,P

6. Appendix (Diagrams)

3.1.1 Frontend Architecture



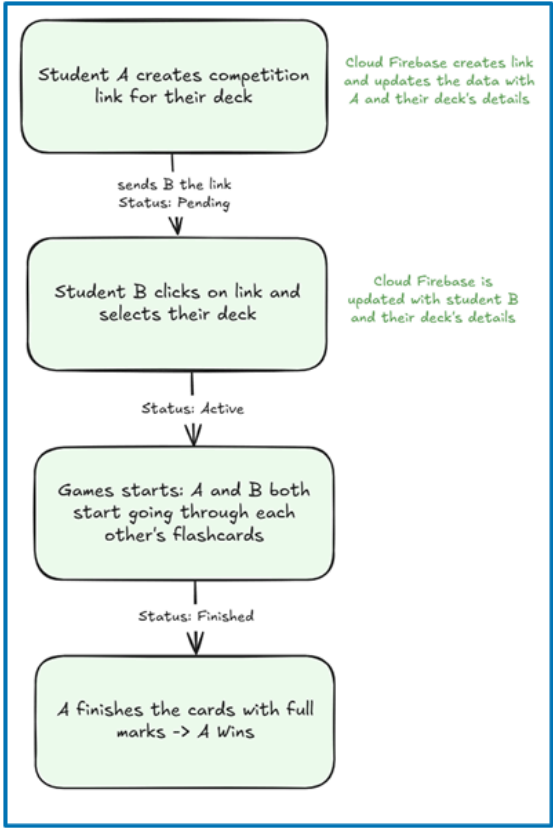
3.1.3 Whiteboard



Whiteboard System Components

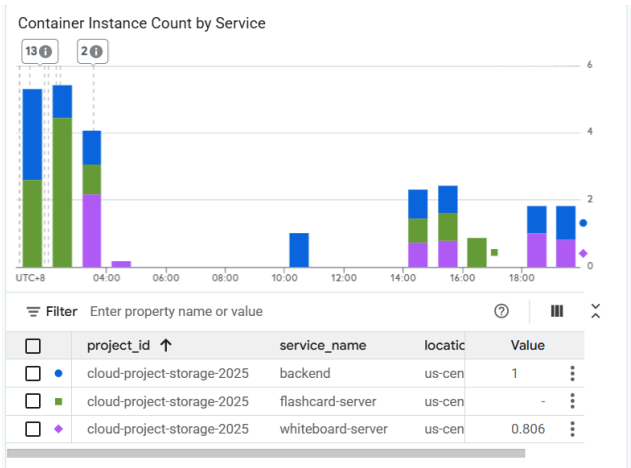
- **Student Dashboard:** Displays all of the user's documents. Open the Whiteboard button to bring the user to the document's whiteboard room.
- **Whiteboard Client:** Uses Fabric.js library to handle drawing on the whiteboard canvas. Eg. Stroke paths, brush tool, eraser tool, brush color. Sends and receives stroke paths data using WebSockets. Locks users from drawing until connection is verified and data is loaded.
- **The WebSocket Server** validates the whiteboard room by checking if it exists on Firestore database, loads up previous stroke paths from the database and broadcasts all drawing strokes to clients in the same room concurrently. It then Save strokes to Firestore database for whiteboard persistence
- **Firestore:** Stores all strokes as a document under the whiteboard-strokes collection of a documentId inside the documents collections in our Firestore database.

3.2.4 Flashcards

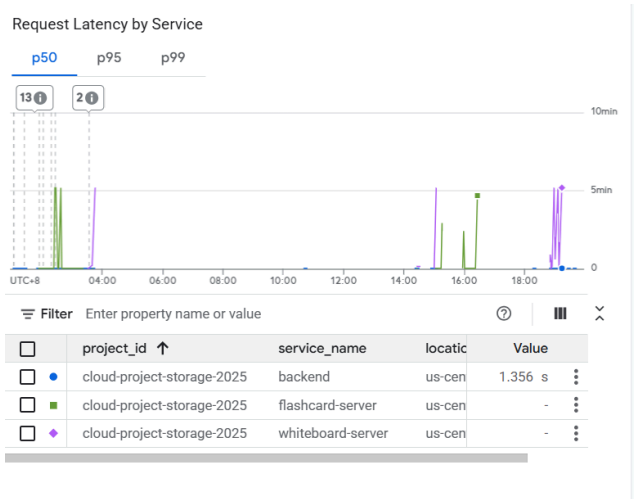


3.3 Evaluation

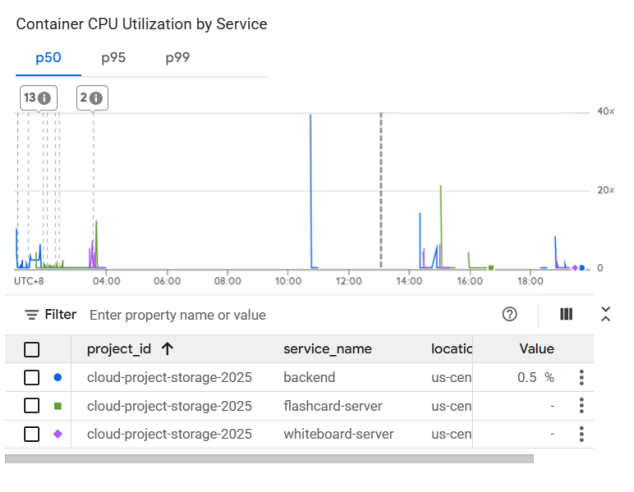
3.3.1 Instance Count



3.3.2: Request Latency



3.3.3: CPU Utilisation



3.3.4: Performance

Performance by Service

Filter Enter property name or value

Service Name	Location	Requests	p50 Latency	p95 Latency	4xx Error %	5xx Error %
backend	us-central1	0.018 /s	0.235 s	0.032 min	0.167 %	0.083 %
flashcard-server	us-central1	0.004 /s	0.015 s	4.935 min	0	9.02 %
whiteboard-server	us-central1	0 /s	32.99 s	5.293 min	6.897 %	0